



9

4

480 sp

نظري

كلية الهندسة المعلوماتية

السنة الأولى

التوابع العودية

د.مدحت الصوص د.هبة حاتم



RB Informatix; 19/12/2024

برمجة اغير

العودية

قلنا أن التابع يُعرّف ثم يُستخدم (أو يُستدعى).
هناك حالة خاصة وهي أن يقوم تابع باستدعاء نفسه. وهذه الحالة تدعى "العودية".

مثال:

```
f1 (n)
{
    .
    .
    .
    f1 ( )
    .
    .
    .
}
```

■ بالنسبة للحاسب كل استدعاء للتابع يعتبر تابع جديد أي عند استدعاء f1 لنفسه في الحاسب تعتبر f1, f2, f3, ...

ملاحظة: كل تابع له ذاكرة خاصة، وكل نسخة من تابع لها ذاكرة خاصة بها.
أي لكل نسخة متحولاتها الخاصة مثلاً عند استدعاء التابع f1(int n) لنفسه فإن النسخة المستدعاة يكون لها متحولها الخاص وليس المتحول ذاته في f1 أي لكل نسخة متحولاتها الخاصة

بعض الأمثلة لفهم فكرة التابع العودي ومعرفة طريقة كتابته:

المثال 1: العامل

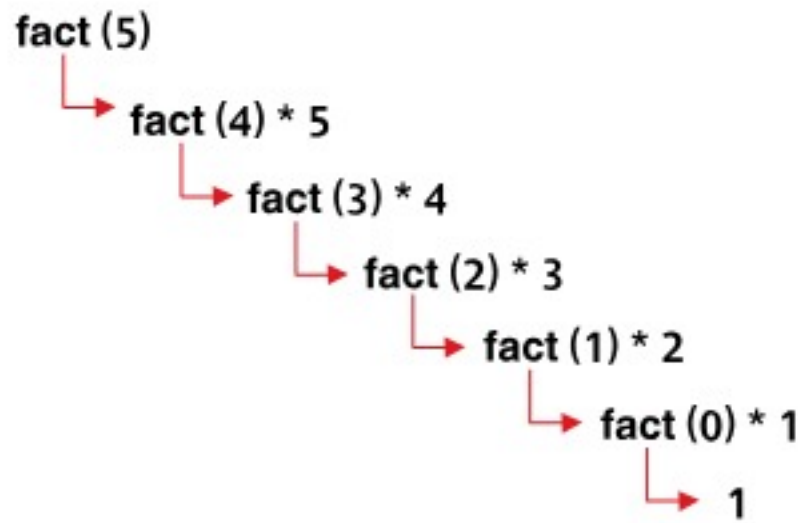
$$n! = 1 \times 2 \times 3 \times \dots \times (n-1) \times n$$

عندما نريد كتابة أي تابع عودي نسال 3 أسئلة ونجيب عنها:

1. ما هو شرط التوقف؟
2. ما هي المسألة الأبسط من نفس النوع؟ (إذا أردت حل 5!، فالسؤال الأبسط الذي من نفس النوع هو 2!).
3. كيف أربط حلاً للمسألة باستخدام حل المسألة الأبسط؟

```
int fact (int n)
{
    if (n == 0)
        return 1;
    return fact (n-1) * n;
}
```

آلية التنفيذ:



شرح المثال 1:

نعلم أن $1 = 0!$ بالتالي فإن شرط التوقف هو أن نصل إلى $n=0$ حيث أن شرط التوقف يؤخذ من أبسط حالة أي 0

ونقوم بحل مسألة أبسط للحصول على حل المسألة المطلوبة

فمثلاً لدينا 5! لحلها نقوم بحل 4! أي أننا نستخدم الحالات الأبسط للحصول على حل الحالة المعقدة $5! = 5 * 4!$

$$4! = 4 * 3 * \dots * 0! = 24$$

نحاول أن نربط حل المسألة البسيطة بالمسألة المطلوبة وذلك عن طريق معلومات موجودة سابقاً وغير ذلك ويكون حسب ما تتطلبه المسألة وفي المسألة لدينا العامل 1 فهو جداء متسلسل من الرقم المطلوب إلى 0 بحيث ينقص 1 من كل عدد عن سابقه أي

$$5! = 5 * 4 * 3 * 2 * 1 * 0!$$

بالتالي نجد أن $n!$ توجد بالعلاقة

$$n! = n * (n-1) * (n-2) * (n-3) * \dots * 0!$$

$$(n-1)! = (n-1) * (n-2) * (n-3) * \dots * 0!$$

$$n! = n * (n-1)! <=$$

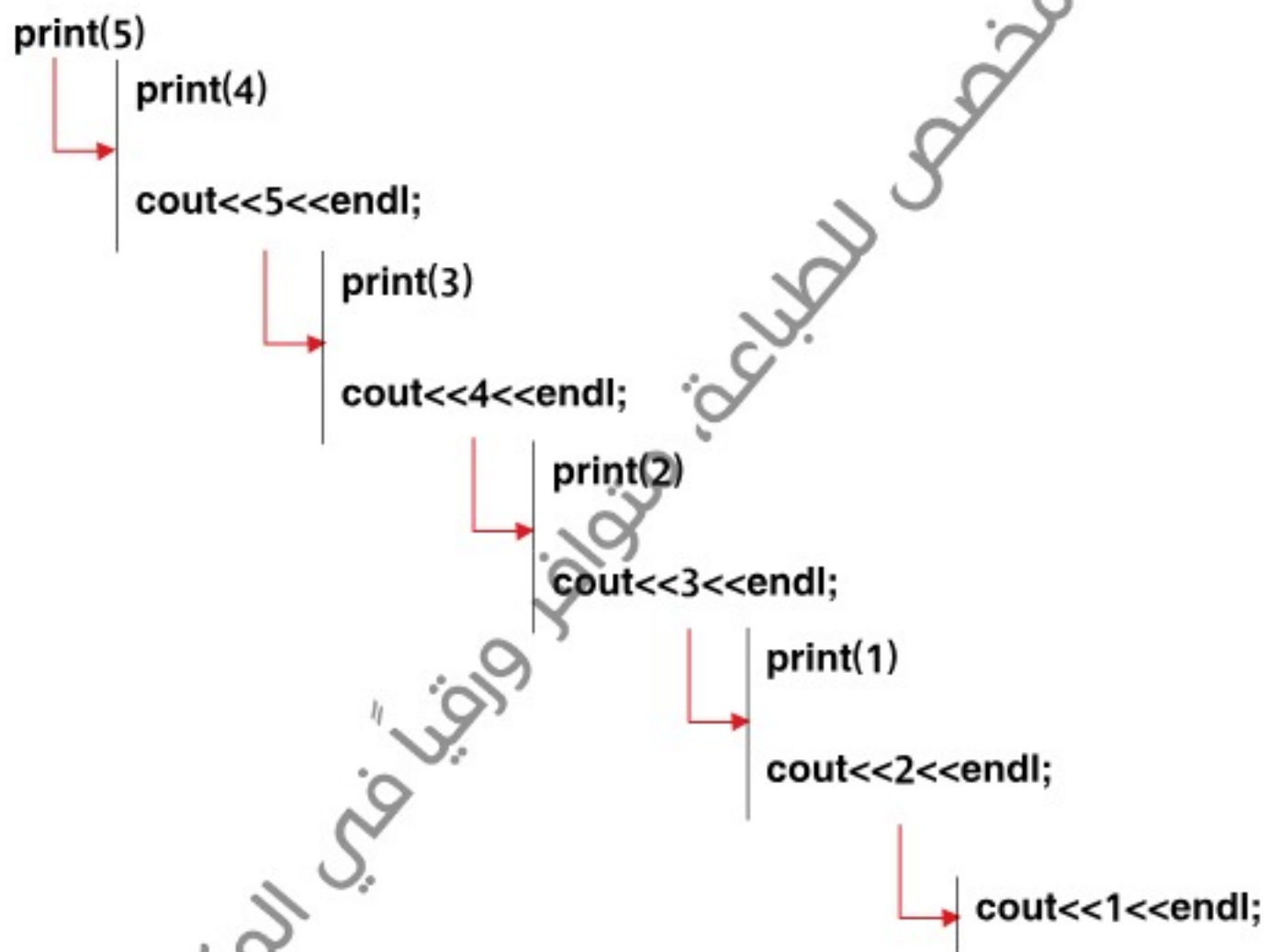
أي عند التطبيق على 5!

$$120 = 24 * 5 = 4! * 5 = 5! <=$$

مثال 2: طباعة الأعداد من 1 إلى n

```
void print (int n)
{
    if (n == 1)
        cout << n << endl;
    else
    {
        print (n-1);
        cout << n << endl;
    }
}
```

آلية التنفيذ:



شرح المثال 2:

وهو طباعة الاعداد بدءا من 1 وصولا إلى n

بما أن أبسط حالة رقم اريد طباعته هو 1 فان $n=1$ هو شرط التوقف بحيث إذا تحقق $n=1$ يطبع 1 اي يطبع n
 لحل المسألة نأخذ حالة بسيطة ونحلها في حال لم تكن $n=1$ نتحقق الحالة المطلوبة فعلا لطباعة الاعداد
 من 1 إلى 5

بما أن $n \neq 1$ نستمر في تجريب الاعداد بحيث كل عدد ينقص عن سابقه بمقدار 1 حتى نصل إلى $n=1$
 وبعد أن نصل إلى $n=1$ فإننا نبدأ بعمليات الطباعة تبعا للاعداد بدءا من 1 بحيث كل عدد يُطبع العدد
 اللاحق له

وعندها يقوم بطباعة الرقم 1 ثم يقوم بطباعة 2 ثم 3 وهكذا بحيث أنه كلما وصل إلى عدد قام بطباعة
 العدد اللاحق له

بالتالي نجد أنه عندما n-2 يطلع n-1 وعندما n-1 يقوم بطباعة n

مثال 3: القاسم المشترك الأكبر "GCD" لعددين

طريقة الحل:

نطرح الصغير من الكبير حتى يظهر نفس العدد عند كل من العددين.

مثال

24	10
14	10
4	10
4	6
4	2
2	2

الحل 2: باستخدام while:

```
int GCD (int x, int y)
{
    while (x != y)
    {
        if (x > y)
            x = x - y;
        else
            y = y - x;
    }
    return x;
}
```

الحل 1:

```
int GCD (int x, int y)
{
    if (x == y)
        return x;
    else if (x > y)
        return GCD (y, x - y);
    else
        return GCD (x, y - x);
}
```

The end

This was the last lecture

See you in the second semester

Good luck

